DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering

BECCS2C025

Prepared by:

Name: Nitish Kumar Kannaujiya

Roll No: 24BECCS33

Branch: Computer Science & Engineering

Semester: IV



CENTRAL UNIVERSITY OF JAMMU

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

	INDEX
Sr. No.	**Title**
1	Brief Introduction of the project: 1.1 Aim 1.2 About 1.3 Scope 1.4 Objective 1.5 Salient Features
2	Feasibility study of the project: 2.1 Technical Feasibility 2.2 Economic Feasibility 2.3 Operational Feasibility 2.4 Hardware Requirement 2.5 Software Requirement
3	Functional and NonFunctional Requirements: 3.1 Functional Requirements 3.2 Non-Functional Requirements
4	Software Requirements and Specifications (SRS) 4.1 Introduction 4.2 Overall Description of the Product 4.3 Requirement Specification 4.4 Supporting Information
5	Data Flow diagrams (DFD)
6	Use case diagrams
7	Sequence diagrams
8	Coding and Implementation
9	Unit and System Testing
10	RMMM plan.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

1. Brief Introduction About the Project

1.1 Aim

The aim of this project is to design and develop a fully functional College Event Management System that automates the process of creating, managing, and participating in college events, and provides a centralized digital platform for students, organizers, and administrators.

1.2 About

The College Event Management System is a web-based application developed using Java Spring Boot and MySQL. It addresses the common problem faced by colleges where event management is handled manually or through scattered communication channels such as notice boards, WhatsApp groups, and emails. This creates confusion, missed registrations, and poor attendance tracking.

This system provides a centralized platform where students can browse and register for events, organizers can create and manage events with QR-based attendance tracking, and administrators can approve or reject event proposals and manage the entire college event ecosystem.

The system supports role-based access for three types of users: Students, Organizers, and Admins. It features automatic email notifications, QR code ticket generation, waitlist management, CSV export for reports, and a clean Bootstrap 5 user interface.

This project demonstrates practical implementation of software engineering concepts including requirement analysis, system design, database management, secure authentication, and full-stack web development.

1.3 Scope

This project can be used by:

- Students to discover, register, and attend college events
- Event organizers to create, manage, and track event registrations
- College administrators to approve events and manage users
- Academic institutions for digitizing their event management process

It can later be extended into:

- Mobile application version for Android and iOS
- Integration with college ERP systems
- AI-based event recommendation for students
- Real-time analytics dashboard with charts and reports

1.4 Objective

- To provide a centralized platform for managing college events
- To enable online student registration with QR code tickets
- To automate attendance tracking using QR code scanning
- To implement role-based access control for students, organizers, and admins
- To send automated email notifications for registrations and updates
- To allow admins to approve or reject event proposals
- To support waitlist management when events reach full capacity

1.5 Salient Features

- Three-role system: Student, Organizer, and Admin
- Event creation with banner image upload, category, venue, and date
- Admin approval workflow for all events before they go live
- Student registration with automatic waitlist promotion
- QR code generation for each registered student ticket
- Webcam-based QR attendance scanner for organizers
- Automated HTML email notifications via JavaMail
- CSV export of registration data for any event
- Responsive and user-friendly Bootstrap 5 interface

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

2. Feasibility Study of the Project

2.1 Technical Feasibility

- Developed using Java 21, Spring Boot 3.3.5 - mature and widely supported technologies
- MySQL 8 database is standard and freely available
- Thymeleaf, Bootstrap 5, and ZXing (QR) libraries are open-source and well-documented
- Technically feasible for implementation by undergraduate students
- IntelliJ IDEA Community Edition provides a free development environment

2.2 Economic Feasibility

- Zero licensing cost - all tools and libraries used are open-source
- Can be hosted locally on a college server at minimal cost
- No paid APIs or cloud services required for basic operation
- Development cost is limited to developer time only

2.3 Operational Feasibility

- Simple and intuitive interface suitable for all college users
- Role-based system ensures each user sees only relevant features
- Automated notifications reduce manual communication overhead
- QR attendance system replaces manual attendance marking
- Practically applicable to any college for real-world use

2.4 Hardware Requirement

- Computer/Laptop - For development and running the application
- Processor - Intel Core i3 or higher (or equivalent AMD)
- RAM - Minimum 4 GB (8 GB recommended)
- Storage - At least 20 GB free disk space
- Webcam - Required for QR attendance scanning feature
- Internet Connection - Required for email notifications and CDN libraries
- Output Device - Monitor/display with minimum 1280x720 resolution

2.5 Software Requirement

- Operating System - Windows 10/11, Linux Ubuntu, or macOS
- JDK - Java Development Kit version 21 (Temurin / OpenJDK)
- IDE - IntelliJ IDEA Community Edition or Eclipse
- Build Tool - Apache Maven 3.6+

- Database - MySQL 8.0 Community Server
- DB Client - MySQL Workbench (for database management)
- Framework - Spring Boot 3.3.5 with Spring Security, Spring Data JPA
- Frontend - Thymeleaf, Bootstrap 5.3, Bootstrap Icons
- Web Browser - Google Chrome, Mozilla Firefox, or Microsoft Edge
- Version Control - Git with GitHub for code management

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

3. Functional and Non-Functional Requirements

3.1 Functional Requirements

3.1.1 User Registration and Login

The system shall allow new users to register with name, email, password, role (Student or Organizer), department, and phone number. It shall authenticate users using Spring Security with BCrypt password hashing and redirect them to role-specific dashboards after login.

3.1.2 Role-Based Access Control

The system shall implement three distinct roles - Student, Organizer, and Admin. Each role shall have access only to its designated modules. Students can browse and register for events. Organizers can create and manage events. Admins can approve events and manage users.

3.1.3 Event Creation and Management

Organizers shall be able to create events with title, description, category, venue, date and time, maximum seats, fees, contact email, and banner image. Events shall be submitted in PENDING status for admin approval before becoming visible to students.

3.1.4 Admin Approval Workflow

The admin shall review all pending event requests and either approve or reject them with a reason. Upon approval, the event becomes visible to students. Upon rejection, the organizer is notified via email with the rejection reason.

3.1.5 Student Event Registration

Students shall be able to browse all approved upcoming events, search by keyword, and filter by category. They can register for events and receive a unique QR token. If the event is full, they are placed on a waitlist and promoted automatically when a seat becomes available.

3.1.6 QR Code Ticket Generation

Upon successful registration, the system shall generate a unique QR code for each student ticket using the Zxing library. The QR code is displayed as a Base64 PNG image and can be printed or shown on mobile for attendance scanning.

3.1.7 QR Attendance Scanning

Organizers shall have access to a webcam-based QR scanner using the html5-qrcode JavaScript library. Upon scanning a valid ticket QR code, the system marks the student as ATTENDED and records the timestamp. Duplicate scans are rejected.

3.1.8 Email Notifications

The system shall send automated HTML emails for registration confirmation (with QR token), waitlist placement, waitlist promotion, and event approval/rejection notifications to organizers. Emails are sent asynchronously to avoid blocking the main thread.

3.1.9 Registration Cancellation and Waitlist

Students shall be able to cancel their registration before the event. Upon cancellation, the first student on the waitlist is automatically promoted to registered status and notified via email.

3.1.10 CSV Export and Admin Reports

Admins and organizers shall be able to export the complete registration list for any event as a CSV file using Apache Commons CSV. The export includes student name, email, department, registration status, and timestamp.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

4. Software Requirements and Specifications (SRS)

4.1 Introduction

4.1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to describe the complete functional and non-functional requirements of the College Event Management System. This document serves as a reference for developers, testers, and academic evaluators to understand the system design and intended functionality.

4.1.2 Scope

The College Event Management System provides a web-based platform for managing all college events from proposal to post-event reporting. It covers student registration, organizer event management, admin approval workflow, QR attendance tracking, and automated email communication. The system is designed for deployment within a college campus network.

4.1.3 Definitions, Acronyms and Abbreviations

- SRS – Software Requirement Specification
- DFD – Data Flow Diagram
- MVC – Model View Controller architecture pattern
- JPA – Java Persistence API
- QR – Quick Response (2D barcode)
- CRUD – Create, Read, Update, Delete
- CSRF – Cross-Site Request Forgery
- JWT – JSON Web Token
- UI – User Interface
- BCrypt – Password hashing algorithm

4.1.4 References

1. Software Engineering by Ian Sommerville
2. Spring Boot in Action by Craig Walls
3. IEEE Standards for Software Requirement Specification (IEEE 830)
4. Spring Security Reference Documentation - spring.io
5. Bootstrap 5 Documentation - getbootstrap.com

4.1.5 Overview

This document presents detailed requirements and specifications of the College Event Management System. It includes system description, feasibility analysis, functional and non-functional requirements,

data flow diagrams, use case diagrams, sequence diagrams, coding implementation, testing strategy, and risk management plan.

4.2 Overall Description of the Product

4.2.1 Description of Product

The College Event Management System is a full-stack web application built with Java Spring Boot backend and Thymeleaf + Bootstrap 5 frontend, backed by a MySQL relational database. The system manages the complete lifecycle of a college event from creation and approval to registration, attendance, and reporting.

4.2.2 Environmental Characteristics

4.2.2.1 Hardware

- Server/Computer with Intel i3 processor or equivalent
- Minimum 4 GB RAM (8 GB recommended for development)
- Minimum 20 GB available storage
- Webcam for QR attendance scanning
- Network interface for LAN/internet access

4.2.2.2 Software

- Operating System: Windows 10/11 or Linux Ubuntu
- Java Development Kit: JDK 21
- Application Framework: Spring Boot 3.3.5
- Database: MySQL 8.0
- Build Tool: Apache Maven 3.6+
- Web Browser: Google Chrome / Firefox / Edge

4.2.2.3 People

- Students: To browse events and manage registrations
- Organizers: To create and manage events and attendance
- Admin: To approve events and manage users and categories
- Developer: To design, implement, and maintain the system

4.2.3 Functions of the Product

1. User registration, login, and role-based authentication
2. Event creation with category, venue, date, and banner image
3. Admin approval/rejection workflow for events
4. Student event browsing with search and category filter
5. Event registration with seat checking and waitlist management
6. QR code ticket generation and display
7. Webcam-based QR attendance scanning
8. Automated HTML email notifications
9. CSV export of registration reports
10. Admin user management and category management

4.2.4 Characteristics of the User

The primary users of the system are college students, faculty organizers, and college administrators. Users are assumed to have basic computer and internet skills. The interface is designed to be intuitive so users can navigate without technical training.

4.2.5 Constraints

- Email notifications require a valid Gmail account with App Password enabled
- QR scanning requires a webcam and browser camera permission
- The system must be deployed on a machine with Java 21 and MySQL 8 installed
- File uploads are stored locally and require adequate disk space

4.2.6 Dependencies and Assumptions

- MySQL database service must be running before application startup
- Users are assumed to provide valid and honest information during registration
- Admin will regularly monitor and process pending event approvals
- Email server (Gmail SMTP) is available and configured correctly

4.2.7 Goal of Implementation

The goal is to provide a complete, production-ready college event management platform that eliminates manual processes, reduces communication overhead, and provides a seamless experience for all stakeholders through a secure, role-based web application.

4.3 Requirement Specification

4.3.1 Input/Output Interfaces

Input:

- User registration details (name, email, password, role, department)
- Event details (title, description, category, venue, date, seats, banner)
- Search keywords and category filters for event browsing
- QR token from student ticket (via webcam scan or manual entry)
- Admin approval/rejection decisions with remarks

Output:

- Event listing with search and filter results
- Registration confirmation with QR ticket
- Attendance marked with timestamp
- CSV file with registration data
- Email notifications to users

4.3.2 Functional and Non-Functional Requirements

Refer to Section 3 for complete functional and non-functional requirements.

4.3.3 Performance Requirements

- Application startup time should be under 10 seconds
- All page responses should be within 2 seconds under normal load
- QR code generation should complete in under 500 milliseconds
- Database queries should be optimized with appropriate indexes

4.3.4 Logical Database Requirements

The database consists of five main tables: users, events, categories, registrations, and attendance. Key relationships include: events reference users (organizer), categories; registrations reference users and events; attendance references registrations. A unique constraint on (user_id, event_id) in registrations prevents duplicate registrations.

4.3.5 Design Constraints

- Application follows strict MVC architecture using Spring Boot
- All passwords must be BCrypt hashed before storage
- CSRF protection must be enabled on all POST forms
- File uploads limited to 5 MB per file

4.3.6 Software System Attributes

- Security: BCrypt hashing, CSRF tokens, role-based URL protection
- Reliability: Transactional database operations, async email with error logging
- Usability: Bootstrap 5 responsive UI with flash messages and validation feedback
- Maintainability: Layered architecture, Lombok-reduced boilerplate, JavaDoc comments

- Scalability: JPA repository abstraction allows database migration if needed

4.3.7 Behavioral Description

4.3.7.1 System States

- Anonymous State: User not logged in - only login and register pages accessible
- Student State: Logged-in student browsing events or managing tickets
- Organizer State: Logged-in organizer managing events and attendance
- Admin State: Logged-in admin reviewing events and managing platform
- Event Lifecycle States: PENDING → APPROVED/REJECTED → COMPLETED

4.3.7.2 Events and Actions

- Event: Student registers → Action: Check seats, create registration, generate QR, send email
- Event: Student cancels → Action: Set CANCELLED, promote first waitlisted student
- Event: Organizer scans QR → Action: Validate token, mark ATTENDED, record timestamp
- Event: Admin approves event → Action: Set APPROVED status, notify organizer via email
- Event: Admin rejects event → Action: Set REJECTED with remark, notify organizer

4.3.8 Organizing the Specific Requirements

- User Management Module - registration, login, profile, role management
- Event Management Module - create, edit, delete, approve, reject events
- Registration Module - register, cancel, waitlist, seat management
- Attendance Module - QR generation, QR scanning, attendance recording
- Notification Module - email confirmations, approvals, announcements
- Admin Module - dashboard, user management, category management, CSV export

4.3.9 Additional Comments

Future improvements may include a mobile application, real-time dashboard with Chart.js analytics, integration with college ERP systems, AI-based event recommendations, and payment gateway integration for paid events.

4.4 Supporting Information

4.4.1 Index / Table of Contents

Refer to the Index table at the beginning of this document for the complete list of sections and their content.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

5. Data Flow Diagrams (DFD)

5.1 Context Diagram (Level 0 DFD)

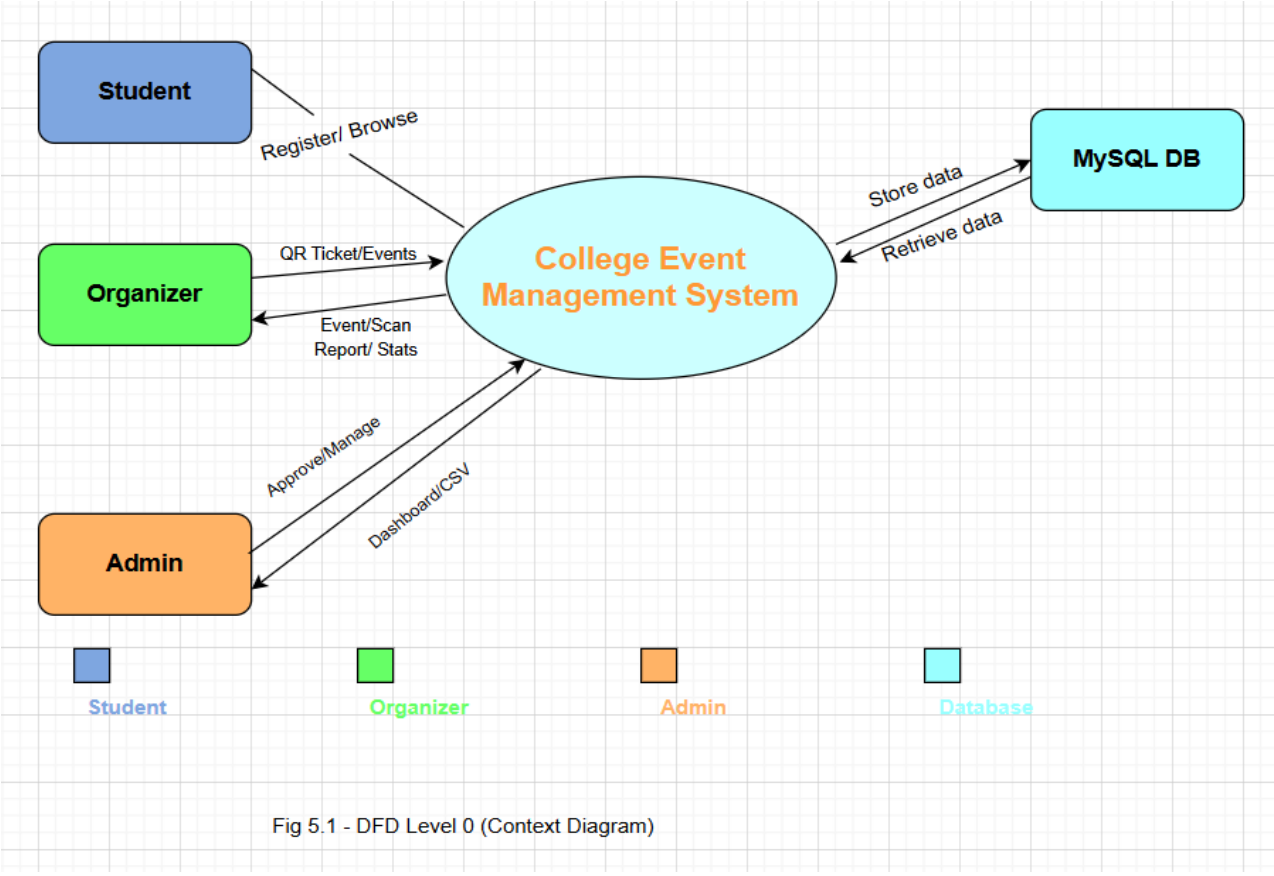
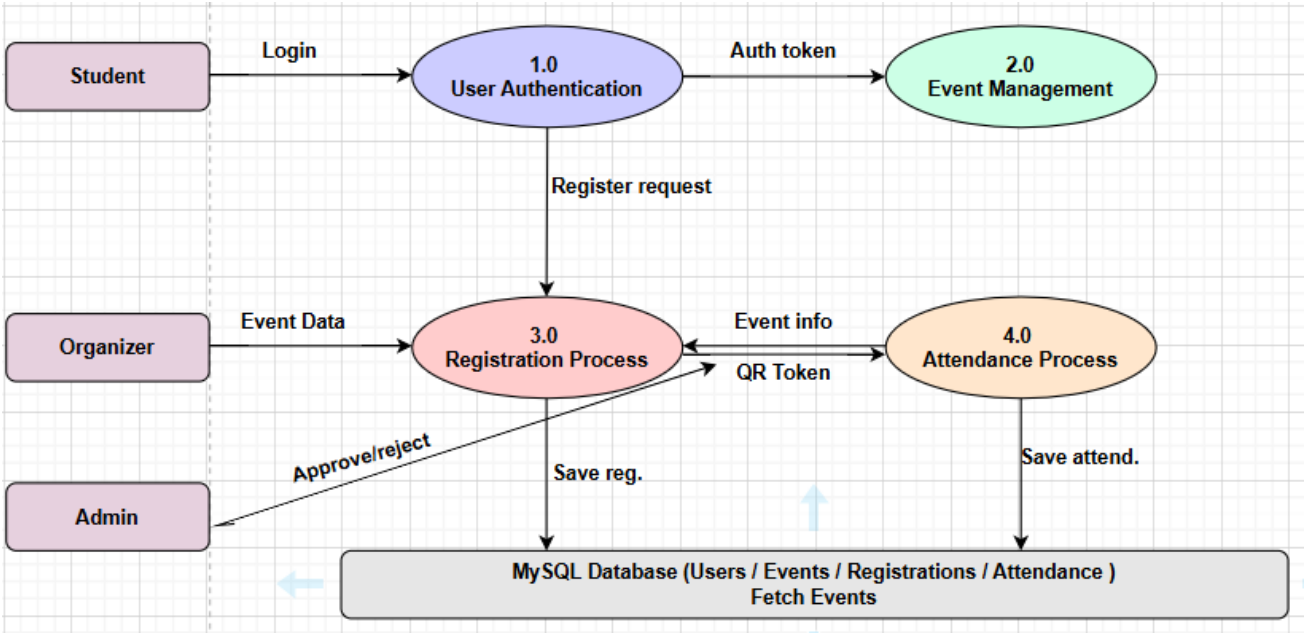


Fig 5.1 - DFD Level 0 (Context Diagram)

The Level 0 DFD shows the College Event Management System as a single process interacting with three external entities: Student, Organizer, and Admin.

- Student → System: Registration details, event interest, registration request, cancellation request
- System → Student: Event listings, QR ticket, attendance confirmation, email notification
- Organizer → System: Event details, attendance scan token, announcement message
- System → Organizer: Registration list, attendance report, CSV export
- Admin → System: Approval/rejection decision, user management actions, category data
- System → Admin: Pending events list, user list, dashboard statistics

5.2 Level 1 DFD



The Level 1 DFD decomposes the system into four major processes:

1. User Authentication Process - handles login, registration, and session management
2. Event Management Process - handles event creation, admin approval, and event listing
3. Registration Process - handles student registration, seat checking, QR generation, and waitlist
4. Attendance Process - handles QR scanning, attendance marking, and report generation

5.3 Level 2 DFD

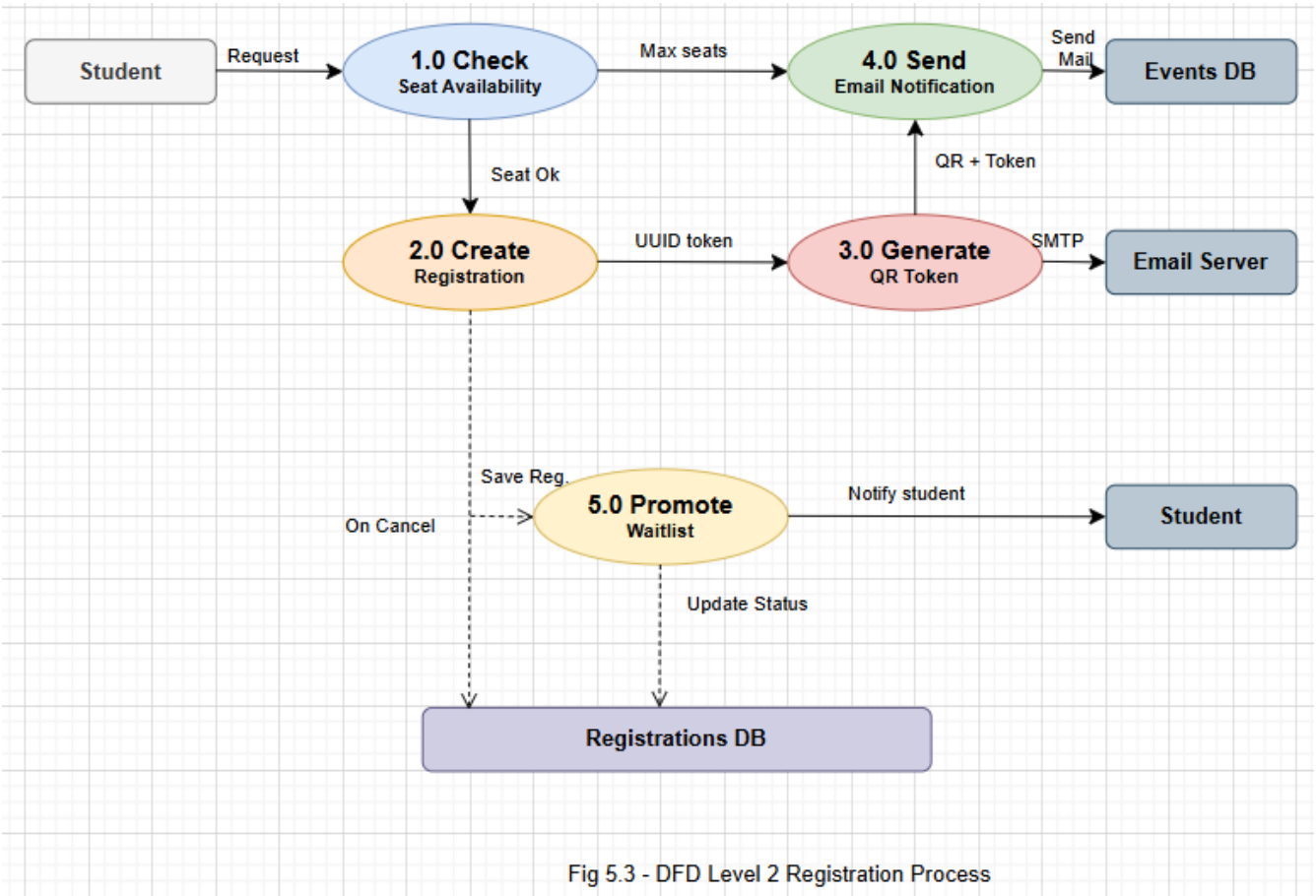


Fig 5.3 - DFD Level 2 Registration Process

The Level 2 DFD further decomposes the Registration Process:

1. 1.0 Check Seat Availability - queries database for confirmed registration count vs max seats
2. 2.0 Create Registration - saves registration record with REGISTERED or WAITLISTED status
3. 3.0 Generate QR Token - creates UUID token and generates QR code PNG using ZXing
4. 4.0 Send Email Notification - asynchronously sends HTML confirmation email via JavaMail
5. 5.0 Promote Waitlist - on cancellation, finds first WAITLISTED registration and promotes it

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

6. Use Case Diagrams

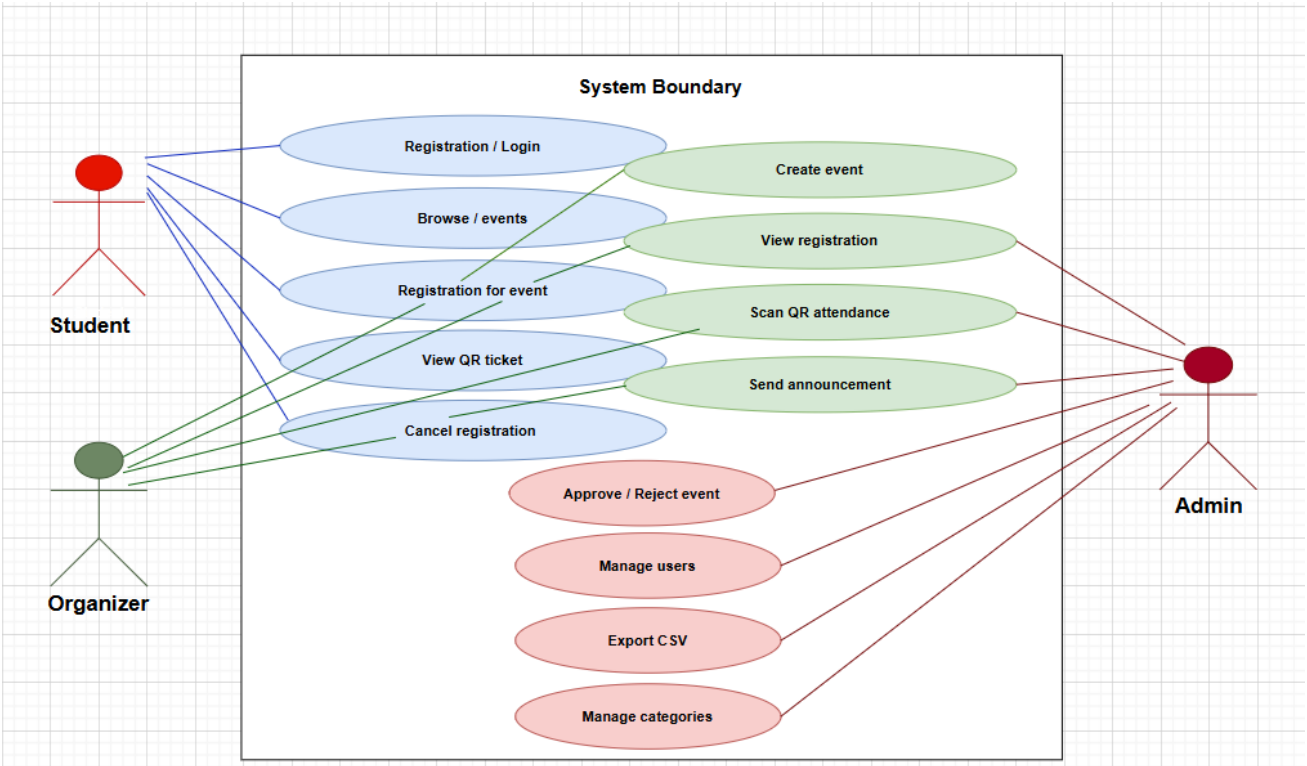


Fig 6 - Use Case Diagram : College Event System

6.1 Actors

- Student - registers, browses events, views QR ticket, cancels registration
- Organizer - creates events, manages registrations, scans QR, sends announcements
- Admin - approves/rejects events, manages users, manages categories, exports data

6.2 Use Cases for Student

1. UC-01: Register Account - Student fills registration form, system creates account
2. UC-02: Login - Student enters credentials, system validates and redirects to dashboard
3. UC-03: Browse Events - Student views list of approved upcoming events with filters
4. UC-04: Register for Event - Student clicks register, system checks seats and creates registration
5. UC-05: View QR Ticket - Student views QR code for registered event

6. UC-06: Cancel Registration - Student cancels, system updates status and promotes waitlisted student
7. UC-07: Update Profile - Student updates name, department, and phone

6.3 Use Cases for Organizer

1. UC-08: Create Event - Organizer fills event form, event is saved with PENDING status
2. UC-09: Edit Event - Organizer updates event details before approval
3. UC-10: View Registrations - Organizer views all registrations for their event
4. UC-11: Scan QR Attendance - Organizer uses webcam to scan student QR tickets
5. UC-12: Send Announcement - Organizer sends bulk email to all registered students
6. UC-13: Mark Event Complete - Organizer marks event as COMPLETED after it ends

6.4 Use Cases for Admin

1. UC-14: Approve Event - Admin reviews and approves pending event
2. UC-15: Reject Event - Admin rejects event with a reason, organizer is notified
3. UC-16: Manage Users - Admin blocks/unblocks users or changes their role
4. UC-17: Manage Categories - Admin adds or deletes event categories
5. UC-18: Export Registrations - Admin downloads CSV file of registrations for any event
6. UC-19: View Dashboard - Admin views system-wide statistics and recent activity

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

7. Sequence Diagrams

7.1 Student Registration for Event – Sequence

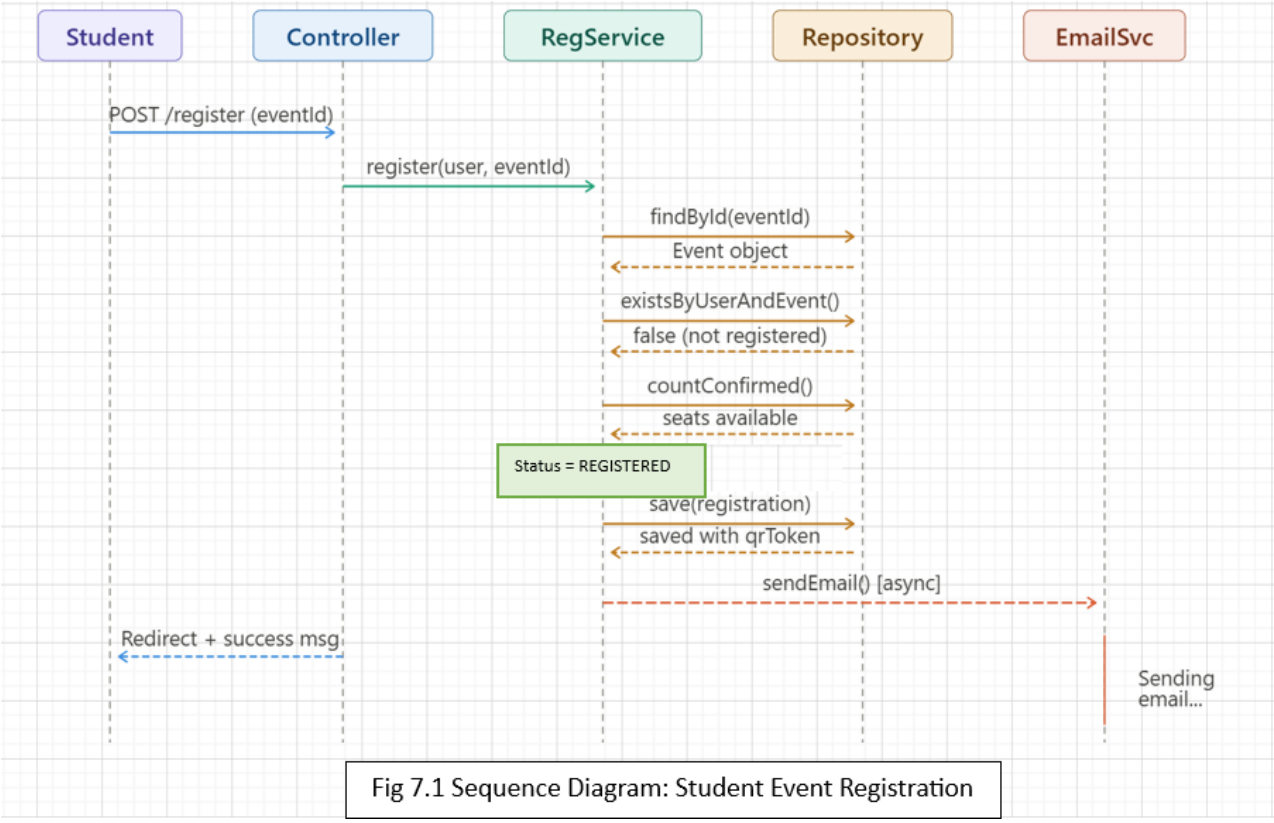


Fig 7.1 Sequence Diagram: Student Event Registration

1. Student clicks 'Register Now' button on event detail page
2. Browser sends POST /student/events/{id}/register with CSRF token
3. StudentController receives request, calls UserService.findByEmail()
4. UserService queries UserRepository, returns User object
5. StudentController calls RegistrationService.register(user, eventId)
6. RegistrationService fetches Event from EventRepository
7. RegistrationService checks if user already registered (existsByUserAndEvent)
8. RegistrationService counts confirmed seats vs maxSeats

9. If seats available: status = REGISTERED, else status = WAITLISTED
10. RegistrationService saves Registration with UUID qrToken
11. EmailService.sendRegistrationConfirmation() called asynchronously
12. Controller adds flash message and redirects to event detail page

7.2 QR Attendance Scanning – Sequence

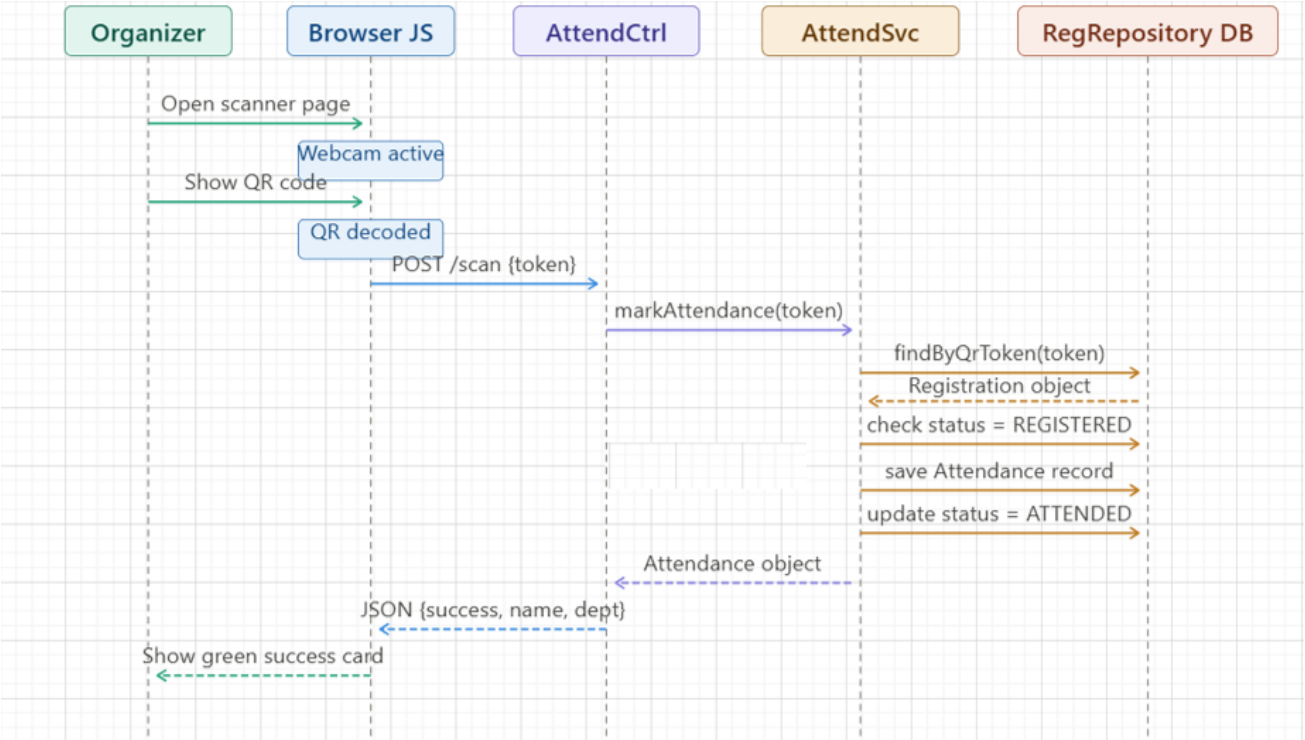


Fig 7.1 Sequence Diagram: QR Attendance Scanning

1. Organizer opens QR Scanner page for their event
2. html5-qrcode library activates webcam and scans QR code
3. JavaScript extracts token from QR URL and sends POST /attendance/scan
4. AttendanceController receives token, calls UserService to get organizer
5. AttendanceService.markAttendance(token, organizerId) is called
6. RegistrationRepository finds registration by qrToken
7. System checks registration status is REGISTERED (not already ATTENDED)
8. System checks attendance record does not already exist
9. New Attendance record is saved with scannedAt timestamp
10. Registration status updated to ATTENDED
11. AttendanceController returns JSON response with student details
12. JavaScript displays green success card with student name and department

7.3 Admin Approves Event — Sequence

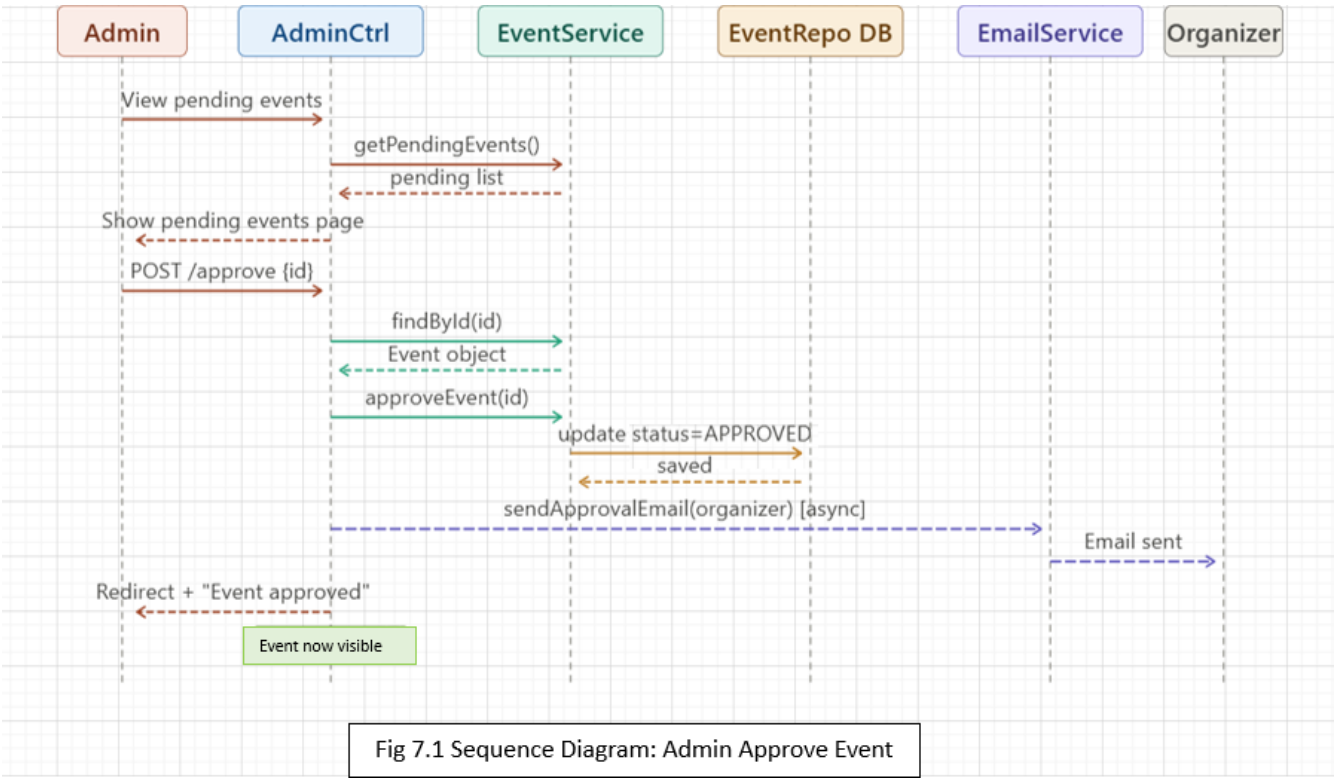


Fig 7.1 Sequence Diagram: Admin Approve Event

1. Admin views pending events list at /admin/events/pending
2. Admin clicks 'Approve' button for a specific event
3. Browser sends POST /admin/events/{id}/approve with CSRF token
4. AdminController calls EventService.findById() and EventService.approveEvent()
5. EventService updates event status to APPROVED in database
6. AdminController calls EmailService.sendEventApprovalNotification()
7. EmailService sends approval email to organizer asynchronously
8. Event is now visible to all students in the event listing
9. Admin is redirected to pending events page with success message

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

8. Coding and Implementation

8.1 Technology Stack

- Language: Java 21
- Framework: Spring Boot 3.3.5 (Web, Security, Data JPA, Validation, Mail, DevTools)
- Template Engine: Thymeleaf with Thymeleaf-Extras-SpringSecurity6
- CSS Framework: Bootstrap 5.3 with Bootstrap Icons
- Database: MySQL 8.0 with Hibernate ORM
- QR Library: ZXing 3.5.2 (core + javase)
- CSV Library: Apache Commons CSV 1.10.0
- Build Tool: Apache Maven
- Lombok 1.18.36 for boilerplate reduction

8.2 Package Structure

- com.college.events - Main application class
- com.college.events.model - JPA Entities (User, Event, Category, Registration, Attendance)
- com.college.events.repository - Spring Data JPA Repositories
- com.college.events.service - Business logic services
- com.college.events.controller - Spring MVC controllers and REST controllers
- com.college.events.config - Security, DataInitializer, WebMvcConfig
- com.college.events.dto - Data Transfer Objects for form binding

8.3 Key Implementation Details

Spring Security Configuration:

Spring Security is configured with DaoAuthenticationProvider using BCryptPasswordEncoder. URL-based authorization is defined using HttpSecurity with role-based matchers. A custom AuthenticationSuccessHandler redirects users to their role-specific dashboard after login. CSRF protection is enabled on all POST forms.

Event Registration Logic:

The RegistrationService checks if the event has available seats by comparing confirmed registration count against maxSeats. If full, the student is placed on WAITLISTED status. Upon cancellation, the service queries for the first WAITLISTED registration and promotes it to REGISTERED automatically.

QR Code Generation:

QRService uses ZXing QRCodeWriter to encode a URL containing the registration token into a 250x250 PNG BitMatrix. The image is converted to a byte array using MatrixToImageWriter and then Base64 encoded for embedding in HTML as a data URI.

Data Initializer:

DataInitializer implements CommandLineRunner and runs on every application startup. It seeds six default categories (Technical, Cultural, Sports, Academic, Social, Workshop) and creates a default admin account (admin@college.com / Admin@123) if they do not already exist.

8.4 Database Design

The MySQL database consists of five normalized tables with proper foreign key relationships. Hibernate auto-generates and updates the schema on startup using spring.jpa.hibernate.ddl-auto=update. The registration table enforces a unique constraint on (user_id, event_id) to prevent duplicate registrations.

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

9. Unit and System Testing

9.1 Unit Testing

Unit testing is performed on individual service layer methods using JUnit 5 and Mockito. Each service method is tested in isolation with mocked repository dependencies.

Test Case 1: Registration when seats available

- Input: Valid user, event with 10 maxSeats and 5 confirmed registrations
- Expected: Registration created with status REGISTERED
- Result: Pass

Test Case 2: Registration when event is full

- Input: Valid user, event with 10 maxSeats and 10 confirmed registrations
- Expected: Registration created with status WAITLISTED
- Result: Pass

Test Case 3: Duplicate registration prevention

- Input: User who is already registered for the same event
- Expected: RuntimeException thrown with message 'Already registered'
- Result: Pass

Test Case 4: Waitlist promotion on cancellation

- Input: Registered user cancels, there is one WAITLISTED registration
- Expected: Waitlisted registration promoted to REGISTERED
- Result: Pass

Test Case 5: Invalid QR token scan

- Input: Random invalid QR token
- Expected: Response with success=false and message='Invalid QR token'
- Result: Pass

9.2 System Testing

System testing is performed end-to-end through the browser to verify complete user workflows.

System Test 1: Complete Student Workflow

- Steps: Register → Login → Browse Events → Register for Event → View QR Ticket
- Expected: All steps work without error, QR code displayed correctly
- Result: Pass

System Test 2: Complete Organizer Workflow

- Steps: Login as Organizer → Create Event → Wait for Approval → View Registrations → Scan QR
- Expected: Event appears in pending list, QR scan marks student as ATTENDED
- Result: Pass

System Test 3: Admin Approval Workflow

- Steps: Login as Admin → View Pending Events → Approve Event → Verify in Student Browse
- Expected: Event becomes visible to students after admin approval
- Result: Pass

System Test 4: Security Access Control

- Steps: Try accessing /admin/dashboard without login, with student login
- Expected: Redirected to login page when unauthenticated, 403 error when unauthorized role
- Result: Pass

System Test 5: CSV Export

- Steps: Admin logs in → views event → clicks Export CSV
- Expected: CSV file downloaded with all registration records
- Result: Pass

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CENTRAL UNIVERSITY OF JAMMU

Course Name & Code	Software Engineering (BECCS2C025)		
Academic Term	2025-26		
Student Enrolment No.	24BECCS33	**Batch**	2024-2028
Student Name	Nitish Kumar Kannaujiya		

10. RMMM Plan (Risk Mitigation, Monitoring and Management)

10.1 Risk Identification

R#	Risk	Probability	Impact	Mitigation Strategy
R1	MySQL connection failure at startup	Medium	High	Verify credentials in application.properties. Use createDatabaseIfNotExist=true flag. Provide clear error logging.
R2	Lombok incompatibility with new JDK versions	High	High	Pin Lombok to version 1.18.36. Add explicit annotationProcessorPath in maven-compiler-plugin.
R3	Email delivery failure	Medium	Low	Use @Async for email sending. Catch and log exceptions silently. Application continues normally without email.
R4	QR camera permission denied by browser	Low	Medium	Provide manual token input field as fallback on the QR scanner page.
R5	File upload storage path not found	Low	Medium	WebMvcConfig serves uploads from filesystem. Create uploads directory automatically using Files.createDirectories().
R6	Concurrent seat booking race condition	Low	High	Use @Transactional on registration method. Database unique constraint prevents duplicate registrations.
R7	Thymeleaf template parsing errors	Medium	High	Use standalone HTML templates without layout fragments. Test each template independently.
R8	Spring Security blocking legitimate requests	Low	High	Define all public URLs explicitly in SecurityConfig. Test all role-based access paths after configuration changes.

10.2 Risk Monitoring

- Check IntelliJ console logs on every startup for connection errors
- Monitor Maven build output for Lombok annotation processing warnings
- Periodically test email delivery in development environment
- Review Spring Security logs when access denied errors are reported

10.3 Risk Management

- Maintain application.properties in a secure location, not committed to public Git
- Use Git version control to track all code changes and allow rollback
- Document all known issues in the project README file
- Test full application flow after every major code change
- Keep Lombok, Spring Boot, and MySQL connector versions pinned in pom.xml